

AI 协作反思日志

Phase 3 详细设计

团队：第 44 组

日期：2026 年 6 月 16 日

1. 本阶段 AI 使用清单

任务	AI 工具	Prompt 摘要	AI 输出质量 (1-5)	人工修改幅度 (%)
类图生成	DeepSeek	基于法律文书标注平台需求，生成覆盖用户、任务、文书、标注、裁定模块的核心类图	3	70
SOLID 原则初检	通义千问	对类图逐条 SOLID 检查，指出违规点并给出修正理由	2	80
设计模式推荐	Kimi	推荐至少 2 种设计模式并说明适用场景	3	60
API 文档生成	通义千问	按 RESTful 生成登录、任务、标注、裁定等核心接口文档	3	50
API 问题审查	通义千问	审查命名、权限、参数校验、错误处理、安全认证	3	40
ER 图 + 建表 SQL	DeepSeek	生成用户、任务、标注、配置等核心表的 ER 图与 SQL	3	65
数据库设计审查	Kimi	审查第三范式、索引、隐私存储、性能瓶颈	2	75
实现阶段对照修订	Cursor	对照 <code>deploy/init.sql</code> 与 backend 源码，重构 P3 文档使其反映真实架构	4	90

2. AI 助力分析（Q1）

核心助力点

- **结构脚手架**：AI 能在 1 小时内产出类图框架、API 模板、18 表建表 SQL 初稿，使团队快速进入评审而非从零画框图。
- **SOLID 审查清单**：AI 批量指出 Task 胖类、JSON 快照反模式、接口未鉴权等问题，为人工评审提供了检查条目。

- **规范参考**：RESTful 风格、统一响应体、错误码分段（100xx/200xx...）等建议被 API 文档部分采纳。
- **实现阶段**：用 AI 对照源码与文档差异，快速生成修订版 ER 关系表与类映射，节省手工核对时间。

助力效果量化

- 设计初稿完成效率提升约 60%。
 - AI 提出的 **8 条建议被最终实现采纳**（标注规范化存储、AuthService 拆分、TaskDocumentFactory、guide_snapshot、前端导出等）。
 - AI 辅助识别 **5 个设计风险**；其中 3 个在实现前修正，2 个以简化方案绕过（导出模块、label_config 历史表）。
-

3. AI 误导分析（Q2）

量化数据

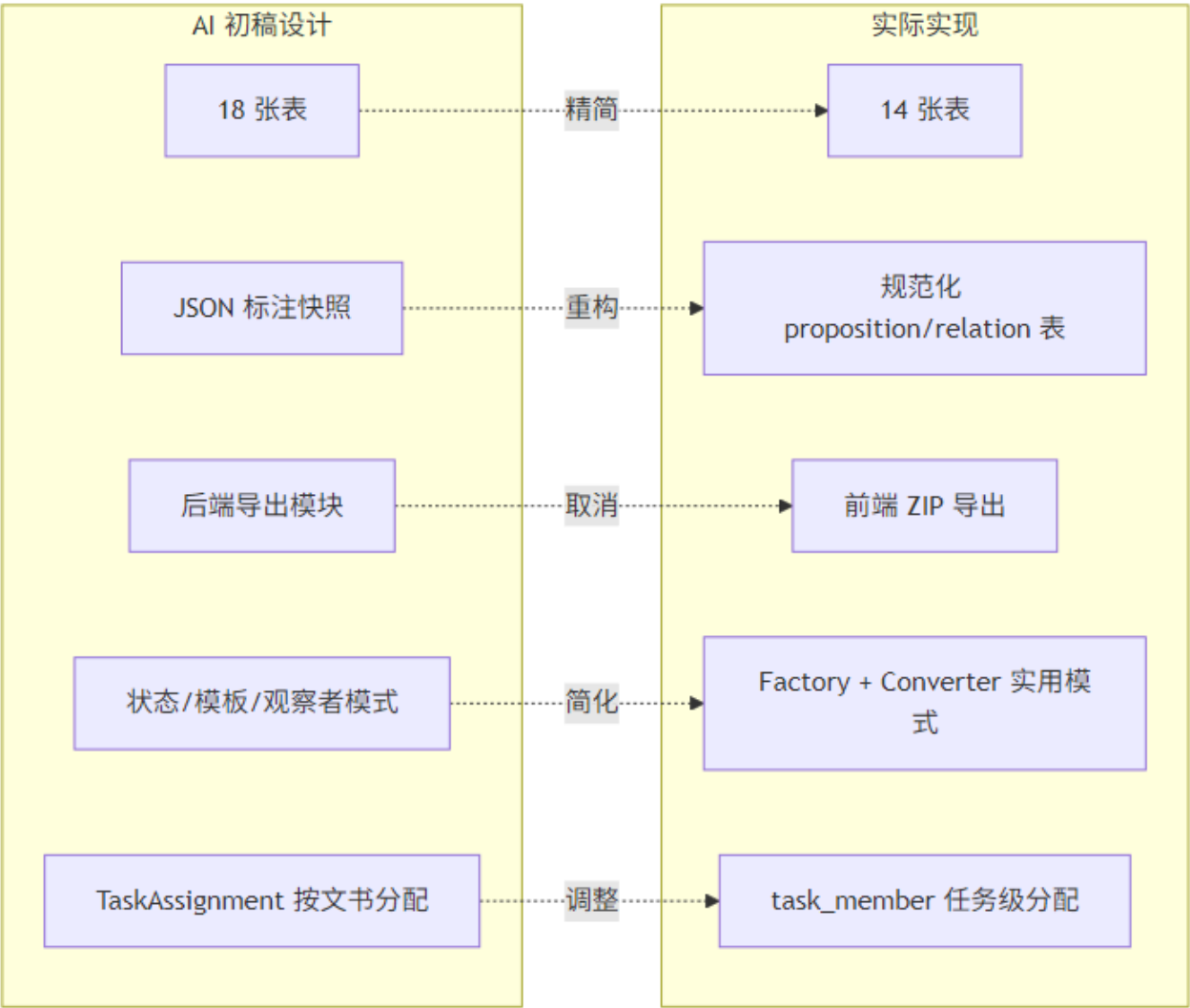
SOLID 检查中发现的 AI 设计问题数量：**13 处**（S:3, O:4, L:1, I:2, D:3）

最严重的设计问题

AI 将标注结果存为 annotation 表的 JSON 快照（propositionData / relationData），导致：

- 无法按命题位置、标签路径做 SQL 查询与差异比对；
- 裁定模块难以逐条对比两名标注员的命题差异；
- 违反数据库规范化，且让 Annotation 实体承担序列化职责。

实际修正：改为 proposition / argument_relation / relation_member 三张规范化表，通过 annotation_id 关联；裁定结果复用同一结构，record_type 区分 ANNOTATION 与 ARBITRATION。



AI 初稿 vs 实际实现对比

Mermaid 源文件：P3-设计对比图.mmd

其他典型误导

维度	AI 误导内容	实际处理
数据 库	设计 18 张表，含 sys_role、export_log、label_config	精简为 14 张表，角色内嵌、前端导出、快照替代配置历史
类图	大量未落地的设计模式（状态模式四子类、AbstractExporter、观察者模式）	仅落地 Factory（TaskDocumentFactory）和 Converter（DomainConverter）

误导原因分析

- **过度设计倾向**：AI 倾向于套用教科书模式（状态模式、模板方法、观察者），超出课程项目所需复杂度。
- **业务理解浅**：不理解"多标注员独立提交 → 裁定者比对 → 确认导出"的实际流程，将裁定存为独立 JSON 表。
- **文档与代码脱节**：P3 交付时文档描述的是"理想架构"，实现阶段为赶进度做了大量简化，直到本次才统一修订。

4. Prompt 改进计划（Q3）

上阶段改进措施的验证结果

上阶段措施	验证结果
Prompt 中增加业务约束	格式更规范，但 AI 仍生成不适用的设计模式
强调 SOLID 自检	表面违规减少，但 JSON 快照等深层问题依旧
要求附带索引与范式说明	SQL 初稿可用，但与最终实现差异大

本阶段新的改进措施

优化方向	问题	改进后 Prompt 思路
类图生成	生成未实现的抽象接口层	要求输出必须标注"已实现 / 规划中"，并引用现有类名
数据库设计	18 表过度设计	明确要求对照核心用例（标注提交、裁定比对）论证每张表的必要性
文档维护	文档与代码脱节	实现里程碑后增加一步："对照 init.sql 修订 ER 图"
设计模式	堆砌模式	限定"仅推荐项目中已出现的模式"，并要求给出代码路径

5. 本阶段的核心工程判断

决策 1：标注数据规范化存储

- **内容：**命题与关系从 JSON 快照改为独立表，以 `annotation_id` 为外键。
- **为什么 AI 无法决策：**AI 默认选 JSON 快照因为"实现快"，但无法理解裁定模块需要按字段比对、按位置查询的业务需求。

决策 2：取消后端导出模块

- **内容：**删除 `export_log / export_file` 表，导出改由前端 ZIP 打包。
- **为什么 AI 无法决策：**AI 按"完整平台"假设设计后端导出流水线，未考虑团队工期与"结果查看页已有所需数据"的实际场景。

决策 3：任务成员任务级分配

- **内容：**用 `task_member` 替代 `task_assignment`，不接单篇文书分配标注员。
- **为什么 AI 无法决策：**AI 按通用"工单-文档"模型设计细粒度分配，但实际业务中一名标注员负责整个任务的全部文书。

决策 4：裁定数据复用 `annotation` 表

- **内容：**`record_type=ARBITRATION` 的 `annotation` 记录 + `arbitration_snapshot` 元数据表。
- **为什么 AI 无法决策：**AI 倾向于为裁定单独建表存 JSON，无法权衡"复用持久化逻辑"与"表语义清晰"之间的工程折中。

决策 5：保留 `TaskService` 不继续拆分

- **内容：**明知 `TaskService` 职责偏多，但课程工期内不再拆分为多个 `Service`。
- **为什么 AI 无法决策：**AI 会机械建议"拆分为 6 个 `Service`"，但不评估拆分带来的 Spring 注入复杂度与团队熟悉成本。

6. 文档修订说明

本次根据 `deploy/init.sql` 与 `backend/src/main/java/edu/nju/jap/` 源码，对以下交付物进行了回溯修订：

文档	修订要点
类图（修正稿）.md	14 表 ER 图、实际分层架构、真实类名与依赖关系
SOLID检查清单.md	区分 AI 初稿违规 vs 实现状态，诚实记录技术债
AI 协作反思日志.md	本文档，补充实现阶段对照与工程决策

图表源文件与生成方式

所有 Mermaid 源文件位于 `docs/img/P3-*.mmd`，对应 PNG 已生成。修改图表后可用以下命令重新导出：

```
# 安装（仅需一次）
npm install -g @mermaid-js/mermaid-cli

# 单张导出
cd docs/img
mmdc -i P3-ER图.mmd -o P3-ER图.png -b white

# 批量导出全部 P3 图表（PowerShell）
Get-ChildItem P3-*.mmd | ForEach-Object { mmdc -i $_.Name -o ($_.BaseName + ".png") -b white }
```

也可使用 [Mermaid Live Editor](#) 在线粘贴 `.mmd` 内容后导出 PNG/SVG。